

UNIVERSIDAD NACIONAL DE INGENIERÍA

Escuela de Ingeniería de Software

SEMANA 05 · GESTIÓN DE PROCESOS

Sincronización de Procesos

Mg. Ing. José Carlos, García La Riva · Sistemas Operativos (SW407) · Escuela de Ingeniería de Software

LOGROS DE APRENDIZAJE

Al finalizar la sesión, el estudiante podrá:

Identificar una condición de carrera (race condition) y la sección crítica que la origina en procesos o hilos concurrentes.

Aplicar semáforos y mutex para coordinar el acceso a recursos compartidos entre procesos/hilos.

Resolver los problemas clásicos de sincronización: productor-consumidor, lectores-escritores y filósofos comensales.

Prevenir interbloqueos (deadlocks) mediante estrategias de asignación ordenada de recursos.

Dos manos en la misma olla

Son las 8:00 p.m. y la cocina de CocinaPerú Express está a tope. Dos cocineros preparan pedidos distintos, pero ambos necesitan retirar el último paquete de queso del almacén.

Ambos revisan el estante, ven "1 unidad disponible" y ambos deciden tomarlo al mismo tiempo. Resultado: uno se queda sin insumo a mitad de la preparación, y el sistema de inventario del restaurante termina marcando "-1 unidades" en el estante.

El problema no fue la falta de insumos: fue que dos procesos leyeron y modificaron el mismo dato compartido sin coordinarse. Hoy aprenderemos a diseñar los mecanismos que evitan que esto vuelva a pasar.



CONCEPTO CLAVE

Condición de carrera y sección crítica

Condición de carrera ocurre cuando dos o más procesos/hilos acceden y modifican un dato compartido de forma concurrente, y el resultado final depende del orden de ejecución (no determinismo).

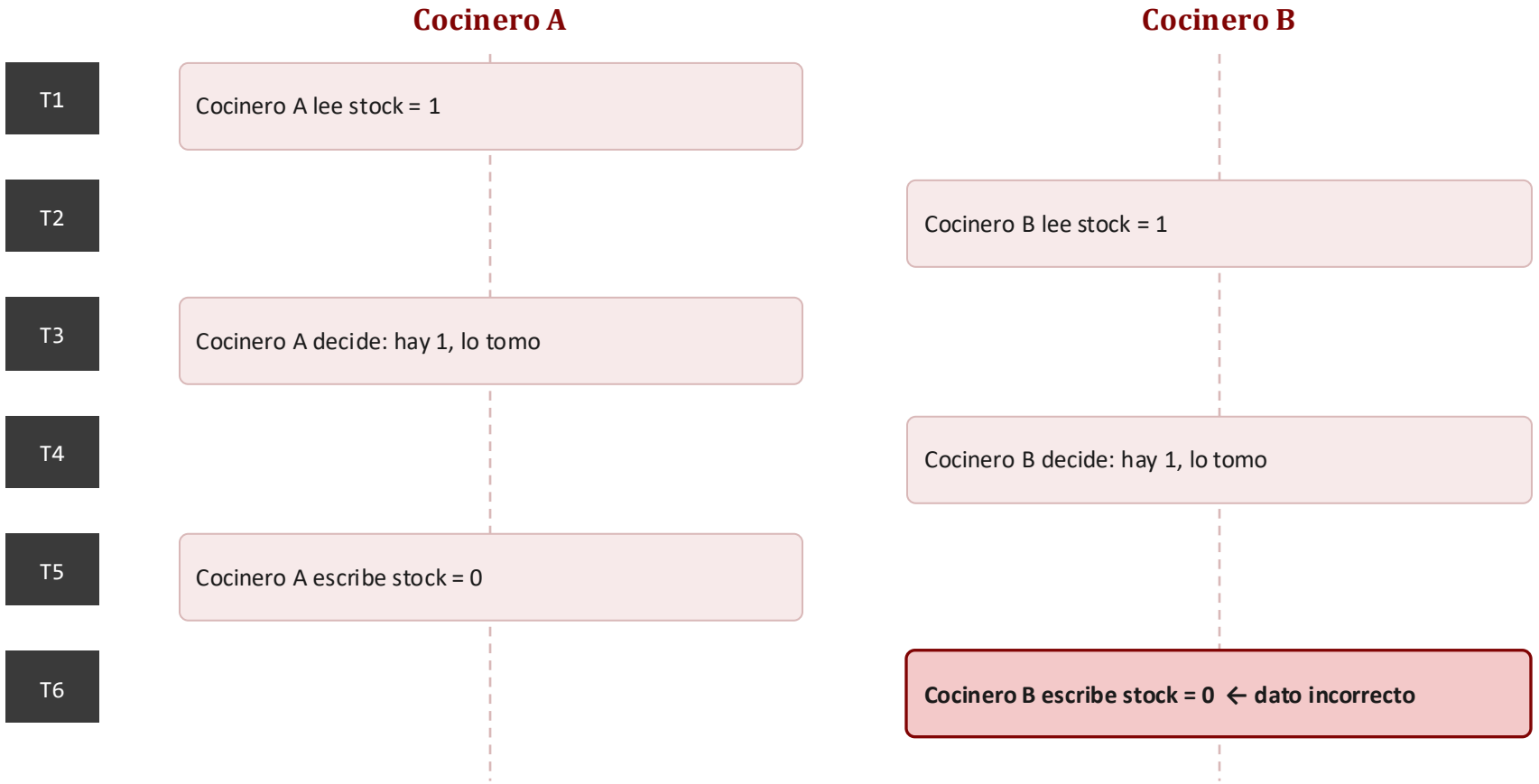
Sección crítica es el fragmento de código donde un proceso accede a un recurso compartido (variable, archivo, estructura de datos) que no debe ser modificado por más de un proceso a la vez.

Exclusión mutua garantiza que, si un proceso está en su sección crítica, ningún otro proceso puede entrar a la suya al mismo tiempo.

Requisitos de una solución exclusión mutua, progreso (nadie fuera de su sección crítica bloquea a los demás) y espera acotada (nadie espera indefinidamente).

CÓMO OCURRE

Anatomía de una condición de carrera



Ambos cocineros intercalan sus operaciones (lectura → decisión → escritura) sin coordinación: el stock final queda mal calculado.

PANORAMA

Tres problemas clásicos de sincronización

Productor-consumidor — uno o más procesos generan datos (productores) y otros los procesan (consumidores), compartiendo un búfer de tamaño limitado — el almacén de insumos de la cocina.

Lectores-escritores — varios procesos leen un mismo dato compartido y otros lo modifican — la pizarra de pedidos que todos los cocineros consultan y el jefe de cocina actualiza.

Filósofos comensales (cocineros comensales) — varios procesos compiten por recursos compartidos dispuestos en círculo — los utensilios que rotan entre las estaciones de la cocina.

PROBLEMA CLÁSICO 1

El almacén de insumos

El almacén de CocinaPerú Express tiene espacio para solo 5 canastas. Los proveedores (productores) dejan canastas de insumos frescos; los cocineros (consumidores) las retiran para cocinar.

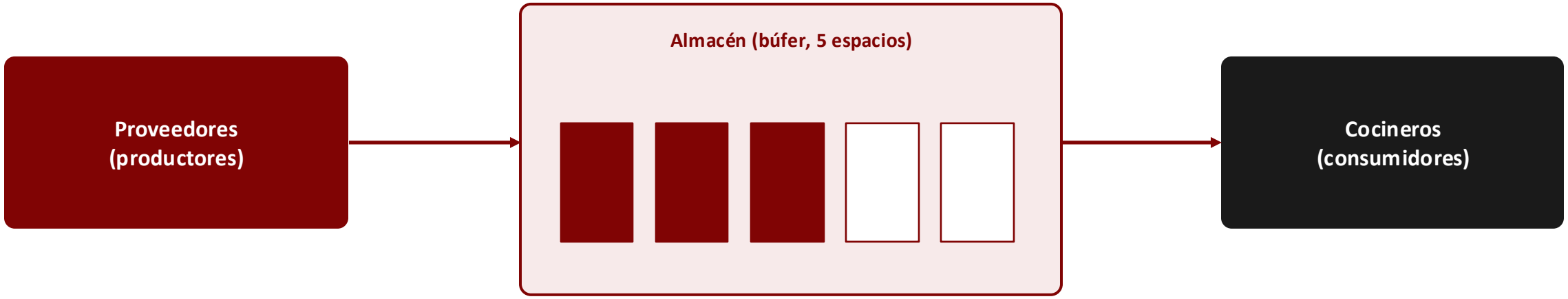
Si un proveedor llega y el almacén está lleno, debe esperar. Si un cocinero busca una canasta y el almacén está vacío, también debe esperar. Y, por supuesto, nadie puede tocar el almacén al mismo tiempo que otro sin chocar.

Este es exactamente el problema del productor-consumidor con búfer acotado, y se resuelve con dos contadores (semáforos) y un candado de acceso exclusivo.



MODELO

Productor-consumidor con búfer acotado



`mutex`	**Semáforo binario (1)**	Exclusión mutua al leer/escribir el almacén
`vacios`	**Semáforo contador (inicia en 5)**	Cuenta los espacios libres del almacén
`llenos`	**Semáforo contador (inicia en 0)**	Cuenta los espacios ocupados del almacén

El productor espera un espacio 'vacío', el consumidor espera un espacio 'lleno'; ambos compiten por el 'mutex' solo durante el acceso al almacén.

Semáforos: la libreta de turnos de la cocina

¿Qué es?

una variable entera controlada solo por dos operaciones atómicas: wait() (o P, decrementa) y signal() (o V, incrementa).

Semáforo binario (mutex)

toma valores 0 o 1; se usa para exclusión mutua, como una sola llave del almacén.

Semáforo contador

toma valores enteros no negativos; representa cuántas unidades de un recurso quedan disponibles (canastas libres, pedidos pendientes).

wait() / signal()

si wait() encuentra el semáforo en 0, el proceso se bloquea hasta que otro proceso ejecute signal() y lo despierte.

Productor-consumidor con semáforos (Python)

```python
mutex   = Semaphore(1)
vacios  = Semaphore(CAPACIDAD)
llenos  = Semaphore(0)
almacen = []

def proveedor(insumo):
    vacios.acquire()
    mutex.acquire()
    almacen.append(insumo)
    mutex.release()
    llenos.release()

def cocinero():
    llenos.acquire()
    mutex.acquire()
    insumo = almacen.pop(0)
    mutex.release()
    vacios.release()
    usar(insumo)
```

Implementación probada con threading.Semaphore: 3 proveedores × 20 insumos, 0 duplicados y 0 pérdidas al consumir.

PUNTOS CLAVE

- vacios.acquire() bloquea al proveedor si el almacén está lleno.
- llenos.acquire() bloquea al cocinero si el almacén está vacío.
- mutex protege el acceso a la lista 'almacen' en sí (exclusión mutua).
- El orden acquire(vacíos/llenos) → acquire(mutex) evita interbloqueo.

PROBLEMA CLÁSICO 2

La pizarra de pedidos

En la cocina hay una pizarra donde se anotan los pedidos activos. Varios cocineros la consultan constantemente (lectores) para saber qué preparar, mientras el jefe de cocina la actualiza cuando llega un pedido nuevo (escritor).

Varios cocineros pueden leer la pizarra al mismo tiempo sin problema. Pero si el jefe de cocina está escribiendo un pedido nuevo, nadie más debe leerla ni escribirla hasta que termine, o alguien vería información a medio actualizar.

Este es el problema de lectores-escritores: maximizar la concurrencia de lectura sin sacrificar la consistencia durante la escritura.



Lectores-escritores: tres políticas posibles

Variante	Regla de acceso	Riesgo
Prioridad al lector	Un escritor debe esperar mientras haya lectores activos, incluso si llegan nuevos lectores.	Inanición (starvation) del escritor si los lectores no dejan de llegar.
Prioridad al escritor	En cuanto un escritor solicita acceso, ningún lector nuevo puede entrar hasta que el escritor termine.	Inanición de los lectores si los escritores llegan con frecuencia.
Acceso equitativo	Se respeta el orden de llegada entre lectores y escritores (cola FIFO de solicitudes).	Mayor complejidad de implementación; requiere una cola de turnos adicional.

En la cocina, una política de acceso equitativo evita que el jefe de cocina (escritor) espere indefinidamente mientras los cocineros no dejan de consultar la pizarra.

PROBLEMA CLÁSICO 3

Los cinco cocineros y los cinco utensilios

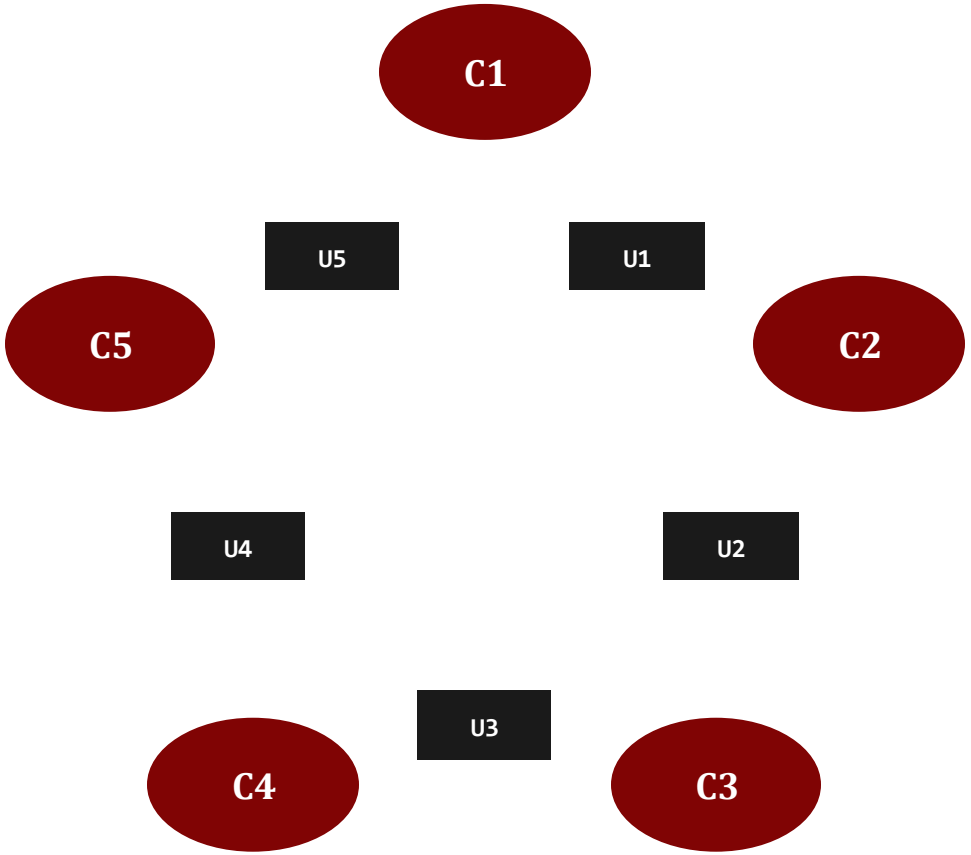
Cinco cocineros trabajan en estaciones dispuestas en círculo. Entre cada par de estaciones hay un solo utensilio especial (una espátula compartida). Para preparar un plato, cada cocinero necesita tomar los dos utensilios a su izquierda y a su derecha.

Si los cinco cocineros toman al mismo tiempo el utensilio de su izquierda, ninguno podrá tomar el de su derecha: todos quedan esperando para siempre. La cocina entera se detiene.

Este es el problema de los filósofos comensales, el ejemplo clásico de interbloqueo (deadlock) por competencia circular de recursos.



Los cinco cocineros en la mesa circular



Situación de deadlock

Cada cocinero Ci toma primero el utensilio a su izquierda (Ui) y espera el de su derecha (Ui+1).

Si los 5 lo hacen a la vez, los 5 utensilios quedan tomados y nadie puede avanzar.

Los cuatro requisitos de Coffman se cumplen: exclusión mutua, retención y espera, no expropiación y espera circular ⇒ deadlock.

COMPARACIÓN DE MECANISMOS

Mutex, semáforo y monitor

Mecanismo	Qué resuelve	Cómo se usa
Mutex (candado)	Exclusión mutua simple sobre un recurso.	lock() / unlock() alrededor de la sección crítica; solo un hilo a la vez.
Semáforo	Exclusión mutua y control de un número limitado de recursos.	wait() / signal(); puede ser binario (0/1) o contador (n unidades).
Monitor + variable de condición	Sincronización de alto nivel: agrupa datos, exclusión mutua y espera condicional en una sola estructura.	El compilador/lenguaje garantiza la exclusión mutua; wait()/signal() sobre la variable de condición gestionan la espera dentro del monitor.

Los monitores (usados en Java 'synchronized' o Python con Condition) simplifican el uso correcto de semáforos, reduciendo errores comunes de sincronización.

Filósofos comensales: romper la simetría

```python
utensilios = [Lock() for _ in range(5)]

def cocinero(i):
    izq, der = i, (i + 1) % 5
    if i == 4:            # el último invierte
        primero, segundo = der, izq
    else:
        primero, segundo = izq, der

    utensilios[primero].acquire()
    utensilios[segundo].acquire()
    preparar_plato(i)
    utensilios[segundo].release()
    utensilios[primero].release()
```

Verificado: 5 cocineros × 30 rondas cada uno, todos completan sus platos sin interbloqueo (asignación asimétrica de recursos).

PUNTOS CLAVE

- Se rompe la espera circular: un cocinero toma primero el utensilio 'contrario'.
- Es la estrategia de asignación ordenada de recursos aplicada al problema.
- Otras soluciones válidas: limitar a 4 cocineros sentados a la vez, o exigir tomar ambos utensilios en una operación atómica.
- El orden de adquisición es la clave, no el número de candados.

TALLER PRÁCTICO

El turno de cierre de CocinaPerú Express

Formen equipos de 3-4 personas. Cada equipo recibe el mismo escenario: implementar el almacén compartido (productor-consumidor) y la mesa de utensilios (filósofos comensales) de CocinaPerú Express, evitando condiciones de carrera e interbloqueos.

1. Implementen el problema del productor-consumidor con semáforos (mutex, vacíos, llenos) en Python o C, con al menos 2 productores y 2 consumidores.
2. Verifiquen, imprimiendo un log de eventos, que no se pierden ni se duplican insumos al finalizar la ejecución.
3. Implementen la solución de los filósofos comensales para 5 procesos/hilos, aplicando la estrategia de asignación asimétrica de recursos.
4. Ejecuten su solución al menos 20 veces seguidas y confirmen que nunca se produce un interbloqueo (todos los procesos terminan).
5. Documenten en un informe breve qué mecanismo de sincronización usaron en cada caso y por qué previene el problema correspondiente.

CIERRE DE LA SESIÓN

Conclusiones

- Una condición de carrera surge cuando procesos/hilos concurrentes acceden a datos compartidos sin coordinación.
- Los semáforos (wait/signal) son el mecanismo base para lograr exclusión mutua y controlar recursos limitados.
- Productor-consumidor, lectores-escritores y filósofos comensales son los tres problemas clásicos que resumen la mayoría de los escenarios reales de sincronización.
- El interbloqueo (deadlock) se puede prevenir rompiendo la espera circular, por ejemplo con asignación ordenada/asimétrica de recursos.

PARA PROFUNDIZAR

Referencias bibliográficas

1. Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts (10th ed.). Cap. 6: Synchronization Tools; Cap. 7: Synchronization Examples.

2. Stallings, W. (2001). Sistemas Operativos (4ta edición). http://williamstallings.com/OS4e.html

3. Tanenbaum, A. S., & Bos, H. Modern Operating Systems. Cap. 2: Interprocess Communication.

4. Documentación oficial de Python — módulo threading (Semaphore, Lock, Condition). https://docs.python.org/3/library/threading.html